

DOS MODELOS PARA REPRESENTAR UNA NEURONA

Ambos modelos representan una **neurona artificial simple**, pero tienen objetivos y comportamientos diferentes.

Característica	TensorFlow Dense(1)	Scikit-Learn Perceptron
Biblioteca	TensorFlow / Keras	Scikit-Learn
Tipo de problema	Regresión o clasificación	Clasificación
Número de neuronas	1 neurona	1 neurona
Entradas	Puede recibir una o varias	Puede recibir una o varias
Activación por defecto	Lineal ($f(x)=x$)	Función umbral
Salida	Valor continuo o probabilidades	0 o 1
Aprendizaje	Descenso por gradiente	Regla del Perceptrón

1. **Perceptrón (Scikit-Learn)** → sirve para introducir:

- pesos
- bias
- error
- clasificación
- frontera de decisión

2. **Dense(1) de TensorFlow** → sirve para introducir:

- neurona artificial
- regresión
- redes neuronales profundas

Decimos que:

Todo Perceptrón es una neurona, pero no toda neurona es un Perceptrón.

Un **Dense(1)** es una neurona general; **Perceptron** es una neurona especializada en clasificación binaria.

EJEMPLOS DE AMBOS

1) **Scikit-Learn**

```
modelo = Perceptron(  
    max_iter=1000,  
    eta0=0.1,
```

```
        random_state=42
    )
```

función de activación escalón:

si $z \geq 0 \rightarrow 1$
si $z < 0 \rightarrow 0$

- para saber pesos y bias usamos los comandos:

`modelo.coef_.shape` y `modelo.intercept_`

2) Tensorflow / Keras

```
capa = tf.keras.layers.Dense(units=1, input_shape=[1])
```

```
modelo = tf.keras.Sequential([capa])
```

Por defecto, `Dense` usa una activación lineal:

```
activation='linear'      #por defecto
```

- para saber pesos y bias usamos los comandos:

```
#pesos y bias de la última neurona
pesos, bias = modelo.layers[-1].get_weights()
```

```
print("Forma de los pesos:", pesos.shape)
#(784, 1)
print("Bias:", bias)
#bias = -2.35
```

```
cantidad de capas
#len(modelo.layers)
```

pesos → determinan cómo influyen las entradas

bias → permite ajustar el punto de decisión

